MINISTRY OF EDUCATION AND TRAINING
----o0o----
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DISTRIBUTED SYSTEMS

Lecturer : Mr. Le Nhu Chu Hiep

HTTP over RPC

Members:

Đặng Thu Huyền	22BA13165
Bùi Mạnh Duy	22BA13097
Đặng Trung Nguyên	22BA13239
Đoàn Đình Khải	22BA13167
Lê Quốc Cường	BA12-036

December, 2025

Contents

I. INTRODUCTION 3
1.1. Overview 3
1.2. Technical Objectives 3
II. SYSTEM ARCHITECTURE 4
3.1. Cloud Infrastructure (Docker) 4
3.2. Monitoring & Load Balancing 4
3.3. Distributed File System (DFS) & Caching 5
3.4. Log Analytics with MapReduce 6
3.5. RPC Interface Design 6
3.6. Concurrency and Synchronization 6
3.7. Observability and Protocol Headers 7
3.8. Request Execution Flow 7
IV.DEPLOYMENT GUIDLINE 8
4.1. Prerequisites Ensure the host machine meets the following requirements before deployment: 8
4.2. Installation & Startup 8
4.3. Usage Manual (End-User) Users can interact with the distributed proxy using two methods: 8
V.TESTING & EVALUATION 9
5.1.Scenario 1: Basic Functionality (End-User Perspective) 9
5.2.Scenario 2: Protocol Transparency & Header Verification 9
5.3.Scenario 3: Distributed Cache Consistency (DFS) 10
5.4.Scenario 4: Fault Tolerance & Self-Healing 11
5.5.Scenario 5: Operational Analytics (MapReduce) 12
VI.CONCLUSION 13

I. INTRODUCTION

1.1. Overview

In the era of Cloud Computing, building systems that ensure both scalability and data consistency is a critical requirement. This project implements a **Distributed HTTP Proxy** based on an extended Client-Server model. The system allows clients to send HTTP requests via a central Proxy Server, which then distributes heavy data-fetching tasks to multiple background Worker nodes using the RPC protocol.

1.2. Technical Objectives

The project aims to design and implement a distributed system that satisfies the following key technical requirements:

1. **Distributed Communication (RPC):** Implement a decoupling architecture where the Proxy Server handles client requests and delegates data-fetching tasks to remote Worker nodes via the **XML-RPC protocol**.
2. **Scalability & High Availability:** The system efficiently handles concurrent requests through **Load Balancing** (Round-Robin algorithm) and horizontally scaling Worker nodes using Docker containers to ensure system availability.
3. **Data Consistency:** Ensure cached data is consistent across all distributed nodes. The system implements a **Distributed File System (DFS)** simulation where all workers share a unified storage volume, preventing data redundancy and ensuring nodes access the latest content.
4. **Data Analytics (MapReduce):** Implement an automated data processing pipeline. The system utilizes the **MapReduce model** to process raw access logs, providing insights into traffic patterns and caching efficiency.

1.3. User Benefits While the system is built on complex distributed architecture, it offers three tangible benefits to the end user:

- **Reduced Latency:** By implementing a Distributed File System (DFS) for caching, the system saves frequently accessed web pages. Subsequent requests for the same content are served immediately from the local cache rather than fetching from the internet, significantly speeding up browsing time.
- **Service Reliability:** The system is designed for High Availability. If a background worker node fails, the Load Balancer automatically redirects the user's request to a healthy node, ensuring uninterrupted service.
- **Transparency:** Users can verify exactly how their request was handled. The system injects custom HTTP headers (X-Worker, X-Cached) into the response, allowing users to see which node processed their data and whether it was retrieved from the cache.

II. SYSTEM ARCHITECTURE

The system operates on the following core components:

- **Proxy Server (Load Balancer):** Acts as the primary gateway, receiving HTTP requests from clients and dispatching them to available Workers.
- **RPC Workers (3 Nodes):** Independent nodes responsible for fetching content from the Internet. They can be scaled up or down without affecting the Proxy.
- **Shared Storage (DFS Simulation):** A shared Docker volume (/app/cache_data) allows all workers to read/write cache files synchronously

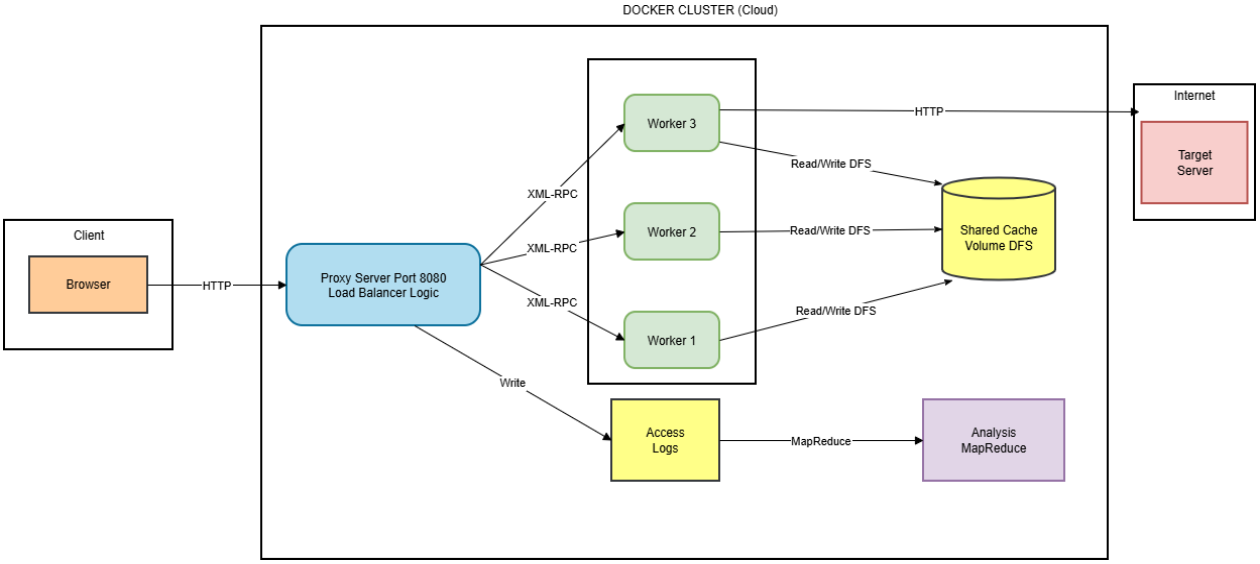

Figure 1: System architecture

III. IMPLEMENTATION DETAILS

3.1. Cloud Infrastructure (Docker)

We utilized docker-compose to orchestrate the environment, establishing a virtual bridge network and deploying four primary containers. Each worker is assigned a unique identifier (WORKER_ID) while sharing network and storage configurations.

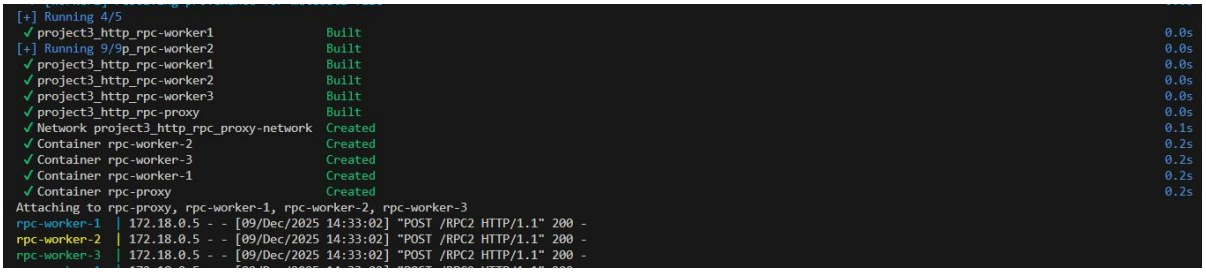

Figure 2: Docker orchestration process successfully initializing the environment and deploying containers.

3.2. Monitoring & Load Balancing

The Proxy Server maintains a dynamic list of active Workers. A background thread performs periodic **Health Checks** every 10 seconds via RPC. If a Worker fails to respond, it is temporarily removed from the dispatch pool to prevent user errors.

```python
class LoadBalancer:
    """Simple round-robin load balancer"""
    def __init__(self):
        self.index = 0
        self.lock = threading.Lock()

    def get_worker(self):
        with self.lock:
            if not active_workers:
                return None
            worker = active_workers[self.index % len(active_workers)]
            self.index += 1
            return worker

load_balancer = LoadBalancer()
```

Figure 3:Code snippet 1: Thread-safe Round-Robin Load Balancer implementation.

```
rpc-proxy      | ============================================================
rpc-proxy      | HTTP PROXY SERVER (RPC-based)
rpc-proxy      | ============================================================
rpc-proxy      | [Proxy] Starting on port 8080
rpc-proxy      | [Proxy] Workers: ['http://worker1:8001', 'http://worker2:8001', 'http://worker3:8001']
rpc-proxy      | [Proxy] Log File: /app/logs/access.log
rpc-proxy      | [Proxy] Worker OK: http://worker1:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker2:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker3:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Active workers: 3/3
rpc-proxy      |
rpc-proxy      | [Proxy] Ready! Use: curl -x http://localhost:8080 http://example.com
rpc-proxy      | [Proxy] Press Ctrl+C to stop
rpc-proxy      |
rpc-proxy      | [Proxy] Worker OK: http://worker1:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker2:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker3:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker1:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker2:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker3:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker1:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker2:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker3:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker1:8001 (Cache: 4 entries)
rpc-proxy      | [Proxy] Worker OK: http://worker2:8001 (Cache: 4 entries)
```

Figure 4: Proxy Server performing periodic Health Checks to monitor the status of 3 Workers.

3.3. Distributed File System (DFS) & Caching

To ensure **Consistency**, the cache is not stored in individual Worker RAM.

- **Write:** When a Worker fetches a webpage, it hashes the URL (MD5) and writes a JSON file to the shared directory shared_cache.
- **Read:** Any Worker receiving a similar request checks the shared directory first. If the file exists (Cache Hit), it returns the content immediately.

3.4. Log Analytics with MapReduce

The analytics.py script implements the MapReduce pipeline on the access.log file:

- **Map:** Parses log lines and emits key-value pairs (e.g., ('domain', 1), ('status', 200)).
- **Shuffle & Reduce:** Groups and aggregates data to generate statistical reports.

```python
def mapper(line):
    """
    MAP phase: Extract key-value pairs from log line
    """
    try:
        parts = line.strip().split('|')
        if len(parts) < 9:
            return []

        timestamp, client_ip, method, url, domain, status, worker, cached, response_time = parts

        results = [
            (('domain', domain), 1),
            (('cache', cached), 1),
            (('worker', worker), 1),
            (('status', status), 1),
            (('response_time', 'total'), float(response_time)),
            (('response_time', 'count'), 1),
            (('hour', timestamp[:13]), 1),
        ]

        return results

    except Exception as e:
        return []
```

Figure 5: Code snippet 2: Mapper function parsing raw logs into key-value pairs.

3.5. RPC Interface Design

To facilitate communication between the Proxy and Workers, we defined a strict XML-RPC interface. Each worker exposes the following methods:

- **fetch_url(url)**: The core function. It accepts a target URL, checks the local consistency cache, and if missing, fetches the content from the Internet. It returns a dictionary containing the HTTP status, headers, and binary content.
- **health_check()**: Used by the Load Balancer to verify worker availability. It returns the current cache size and a timestamp.
- **clear_cache()**: A management function to manually invalidate the cache for consistency testing.
- **get_stats():** Retrieves worker statistics, including cache type, directory path, and the number of cached entries, used for system monitoring.

3.6. Concurrency and Synchronization

To handle multiple concurrent client requests, the Proxy Server utilizes the ***ThreadedHTTPServer*** class. This allows the proxy to process a new request in a

separate thread without blocking incoming traffic. Crucially, to ensure thread safety in this distributed environment, we implemented **Mutex Locks (threading.Lock)** in two key areas:

- **Load Balancer:** To safely increment the round-robin index when multiple threads request a worker simultaneously.
- **Worker Cache:** To prevent race conditions when reading/writing to the in-memory cache dictionary (**cache_lock**).

3.7. Observability and Protocol Headers

To allow the client to verify which node processed their request (transparency), the Proxy Server injects custom HTTP headers into the response before returning it to the client:

- **X-Proxy**: Identifies the server type (RPC-Proxy).
- **X-Worker**: Reveals the specific RPC worker URL that handled the fetch (e.g., http://localhost:8001).
- **X-Cached**: A boolean flag (**True/False**) indicating if the data came from the cache or a fresh fetch.

3.8. Request Execution Flow

This diagram illustrates the complete lifecycle of a client request, demonstrating the interaction between the Proxy, Load Balancer, RPC Worker, and the Shared DFS Cache.

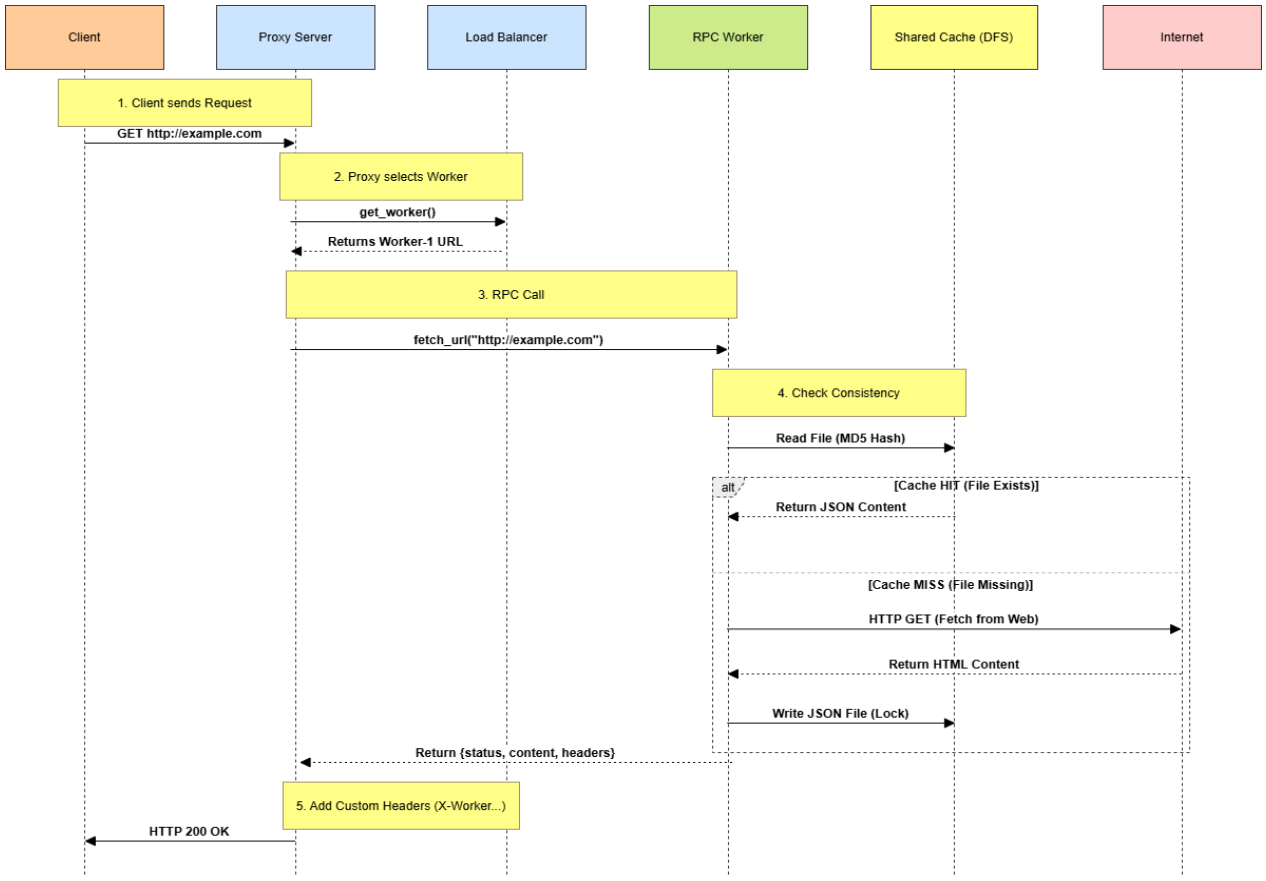

Figure 6: Flow

IV.DEPLOYMENT GUIDLINE

4.1. Prerequisites Ensure the host machine meets the following requirements before deployment:

- **Software:** Docker Desktop (v4.0+), Git.
- **Network:** Ports 8080 (Proxy) and 8001-8003 (Workers) must be available.
- **Operating System:** Windows 10/11, Linux, or macOS.

4.2. Installation & Startup

1. **Navigate to Project Directory:** Open a terminal/PowerShell and move to the project folder:

```
cd Project3_HTTP_RPC
```

2. **Launch the Cluster:** Build and start the container orchestration in detached mode:

```
docker-compose up --build -d
```

3. **Verify Status:** Ensure all 4 containers are active (1 Proxy, 3 Workers):

```
docker ps
```

Success Criteria: The rpc-proxy container must be listening on 0.0.0.0:8080.

4.3. Usage Manual (End-User) Users can interact with the distributed proxy using two methods:

- **Method 1: Command Line (Recommended for Verification)** Use curl to send requests and inspect custom protocol headers (X-Worker, X-Cached):

```
curl -v -x http://127.0.0.1:8080 http://example.com
```

- **Method 2: Web Browser Configuration**
    1. Go to **System Settings > Network & Internet > Proxy**.
    2. Enable **Manual Proxy Setup**.
    3. Set Address: 127.0.0.1, Port: 8080.
    4. **Important:** Ensure "Use setup script" is **OFF** to avoid conflicts.
    5. Open a browser and navigate to HTTP sites (e.g., http://example.com).
    - *Note: The system supports standard HTTP. HTTPS traffic (port 443) is currently not tunneled.*

4.4. Operational Tools

- **Run Automated Test Suite:** Execute the Python test script inside the container environment:

```
docker-compose exec proxy python test_proxy.py
```

- **Generate Analytics Report:** Run the MapReduce job to analyze access logs:

```
docker-compose --profile tools run analytics
```

V.TESTING & EVALUATION

5.1.Scenario 1: Basic Functionality (End-User Perspective)

- **Objective:** Verify that the system transparently proxies HTTP requests and renders content to the user.
- **Procedure:** Configured the browser proxy to *127.0.0.1:8080* and accessed *http://example.com*.

Result:

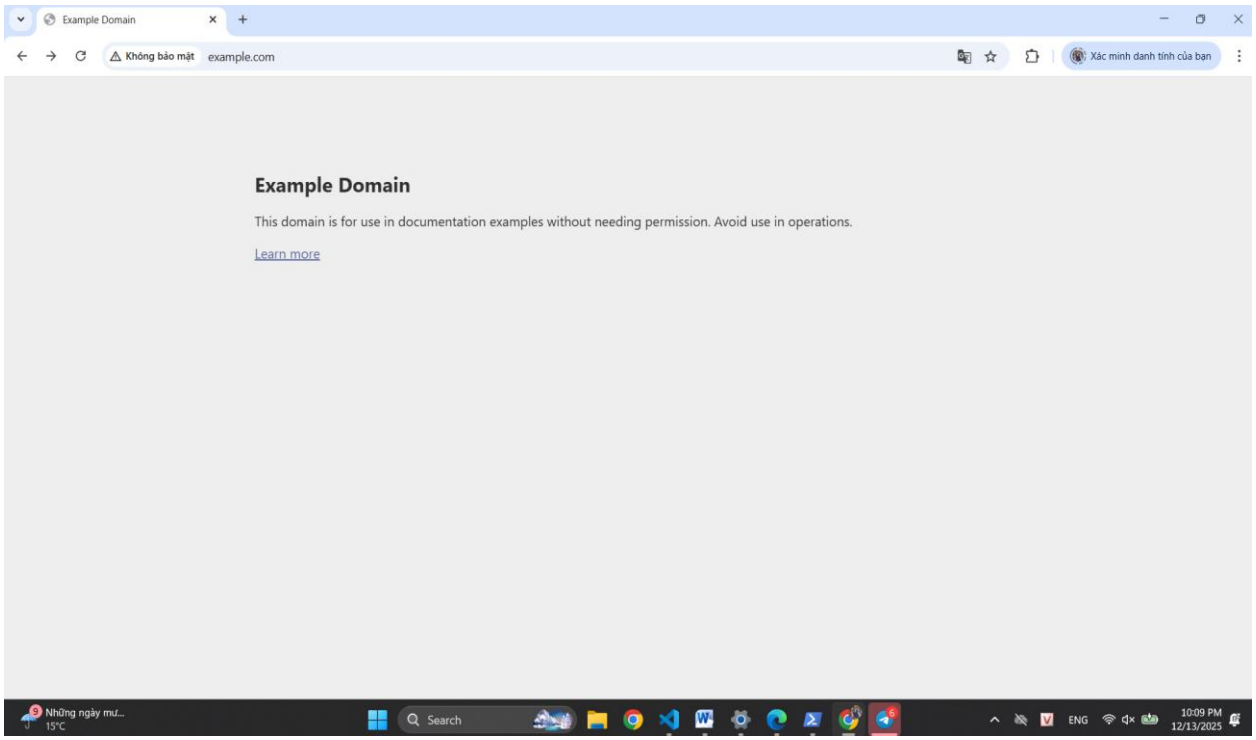

Figure 7:Successful page load via Distributed Proxy.

- **Evaluation:** The browser successfully received the HTML content. The system correctly forwarded the request to a worker node and returned the response to the client without error.

5.2.Scenario 2: Protocol Transparency & Header Verification

- **Objective:** Confirm the injection of custom headers for system observability.
- **Procedure:** Sent a verbose request using *curl*.

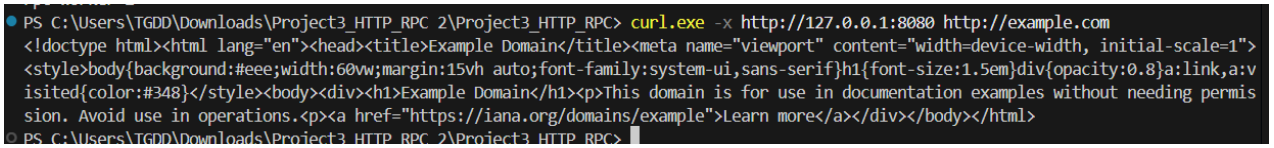

Figure 8: CLI response showing custom HTTP headers

- **Evaluation:**

X-Proxy: RPC-Proxy: Confirms the request passed through our server.
X-Worker: worker-1: Identifies the specific node that processed the task.
X-Cached: False: Indicates a fresh fetch from the Internet.

5.3.Scenario 3: Distributed Cache Consistency (DFS)

- **Objective:** Verify that multiple workers share the same cache (DFS).
- **Procedure:** Executed the *test_proxy.py* script to send sequential requests.

Result:

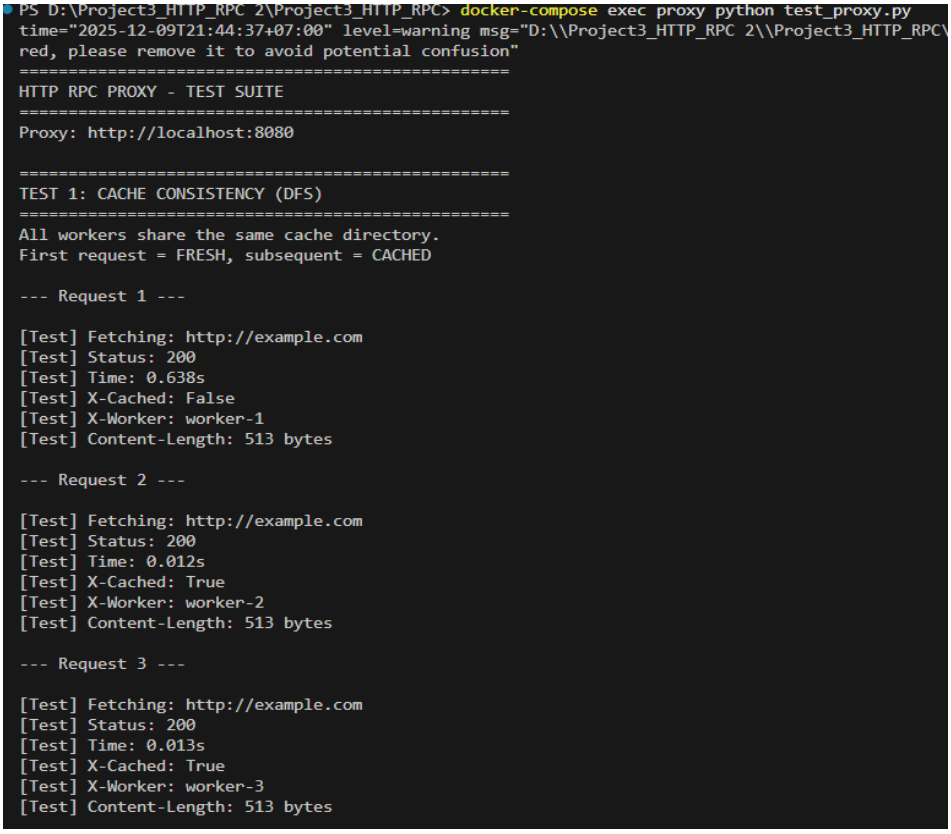

Figure 9: Cache Vertification

- **Evaluation:**

- Fresh Fetch: The first request took **0.638s** (fetched from Internet).
- Cache Hit: The second request took only **0.012s** with *X-Cached: True.*

- Conclusion: The response time was reduced by over **98%**. The logs confirm that subsequent requests were served directly from the shared DFS cache, proving data consistency across the distributed system.

5.4.Scenario 4: Fault Tolerance & Self-Healing

- **Objective:** Verify system stability when a node fails and its ability to recover.
- **Procedure:**

- Fail: Manually *docker stopped rpc-worker-2* (docker stop).

- Test: Sent requests to the proxy.

Result:

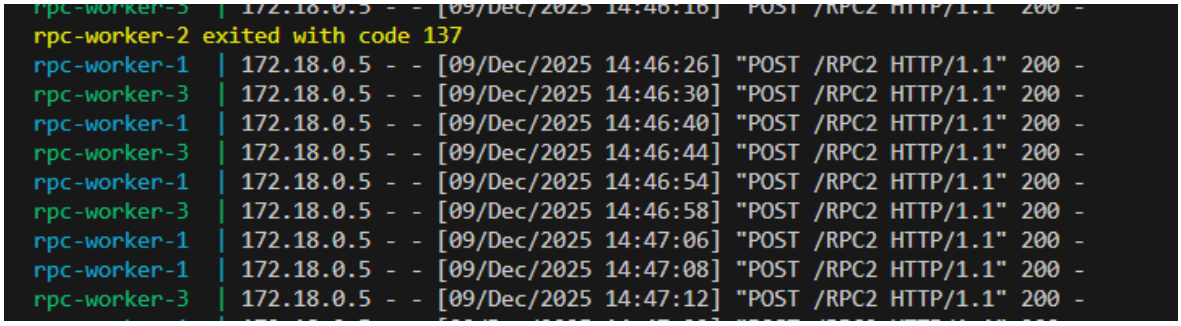

Figure 10: Fault Tolerance Verification - Worker 2 automatically removed from pool after exiting with code 137.

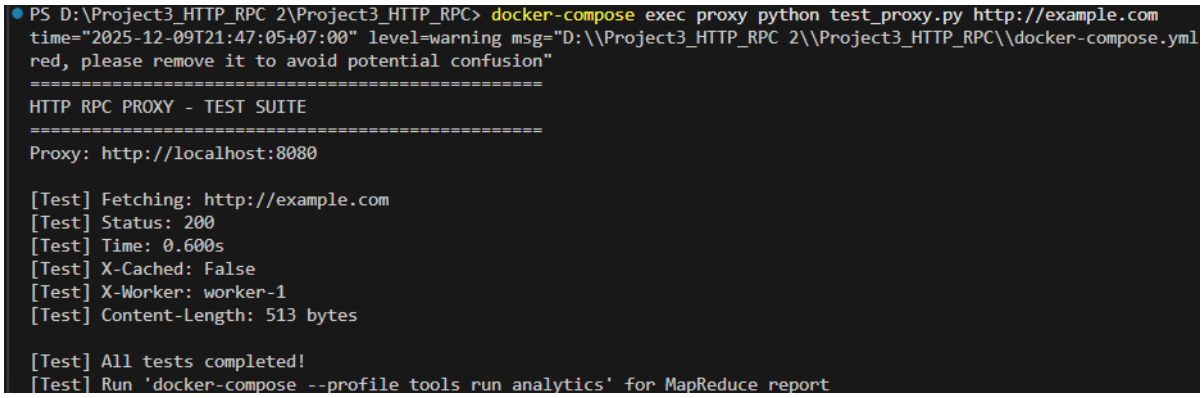

Figure 11:Test results after stopping Worker-2. The system successfully returned HTTP 200 OK via Worker-3.

- **Evaluation:** The Load Balancer automatically redirected traffic to remaining *workers (1 & 3).* No downtime was observed.

Then recovering Worker2 : use *docker start rpc-worker-2*

Figure 12: Recover Worker2.

5.5.Scenario 5: Operational Analytics (MapReduce)

- **Objective:** Analyze traffic patterns.
- **Procedure:** Run the MapReduce analytics tool on the accumulated logs.

Result:

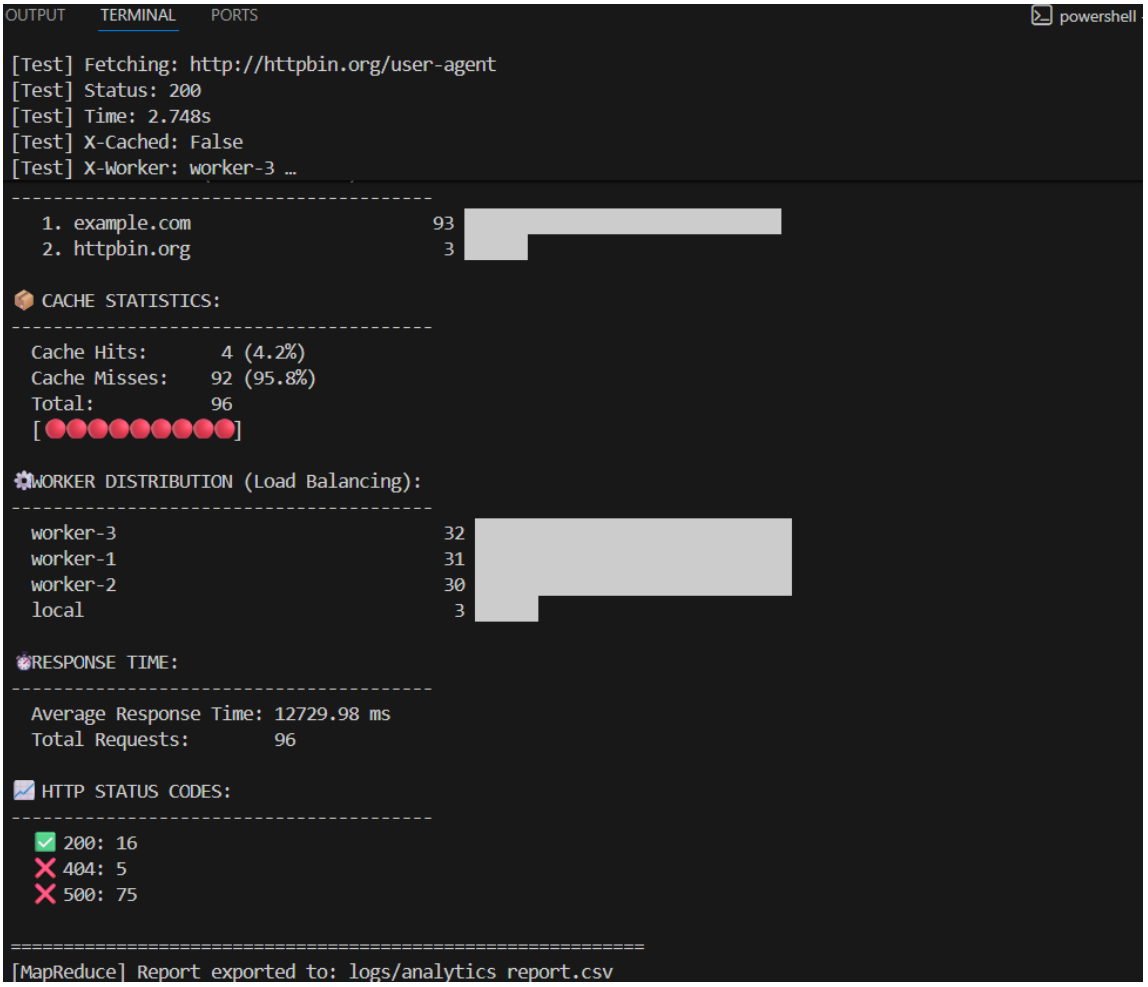

Figure 13 : Analysis tool

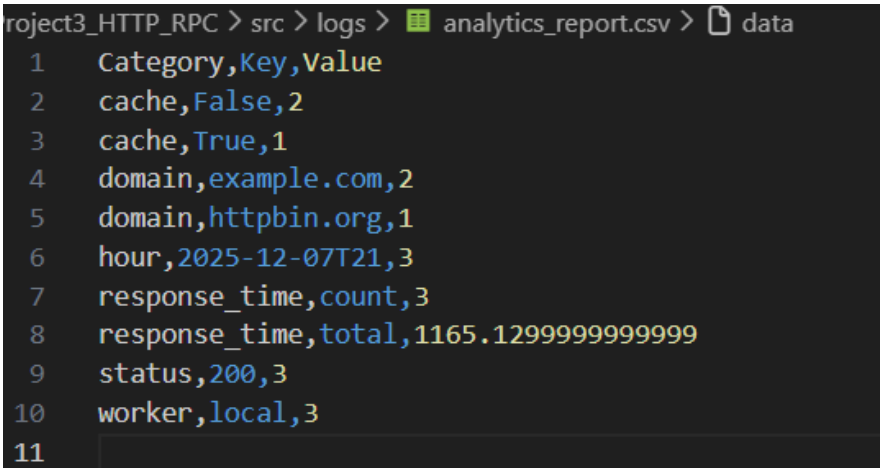

Figure 14: Traffic Analysis Report

Evaluation: The tool successfully aggregated data, showing "example.com" as the top domain and calculating a precise Cache Hit Rate based on the test run.

VI. CONCLUSION

The system successfully met its technical objectives: deploying a complete Distributed Proxy architecture on Docker, ensuring data consistency via DFS, and demonstrating high fault tolerance. The MapReduce mechanism provided valuable insights into system operations. Future improvements include HTTPS support and optimization of the load-balancing algorithm.